Project Milestone 1: Verifiable Tasks

COMP 480/580, Fall 2026

Due: October 31

1 Introduction

AI agents use tools to retrieve information, process data, and create files. An agent's claim that it has finished a task, however, does not show that its result is correct. To evaluate an agent reliably, we need a precise task specification, a reproducible environment, and an executable verifier that checks the actual result. In this milestone, you will create an original task, its execution environment, and a verifier that checks its outcome. Use the public starter below to build and validate your task locally, then publish it on RUSH Lab.

2 Tasks

1. **Task:** Define its inputs, tools, outputs, and success conditions. Renaming the starter is not enough.
2. **Environment:** Include the data, dependencies, and reproducible setup instructions.
3. **Verifier:** Check actual results. Test correct, incorrect, missing, malformed, and boundary cases. **The verifier must not call an LLM.** You may use an LLM to help write it, but the verifier itself is a program that runs without one.
4. **Validation and publication:** Test, run, and publish the task, and provide reproducible evidence.

3 Instructions

3.1 Set up your plugin

Install Git, Python 3.12 or later, and `uv`, then clone the public Hello World starter:

```
git clone https://github.com/zeyuyang8/mochi-hello-world.git my-task
cd my-task
uv sync --extra dev
uv run pytest
```

Replace `my-task` with a unique name. Following the starter's README, adapt the generated manifest, contract, tools, and verifier. Keep the required package structure, and keep reports and run outputs outside the plugin folder. For a worked example, see the arXiv Search and Source Downloader plugin. Its README and source show how the task, tools, environment, and verifier fit together. Use it as a reference while you develop your own original task.

In an agent task, tools expose the inputs and save the agent's answer to a workspace file; the agent does the reasoning. An optional `reference` tool computes the answer key before the agent runs, and it must not be one of the agent's task tools. The verifier receives this pinned reference. If you have no reference tool, set `reference: null` and explain how correctness is established.

3.2 Implement and validate

The verifier implements `verify_input`, `verify_output`, and `verify_task`. The first two check tool calls; the third checks the completed task. Each returns `valid`, a `score` in $[0, 1]$, a `message`, and `details`. Follow the starter's signatures and typed inputs. The verifier must be deterministic: no LLM calls, network access, randomness, subprocesses, or file writes.

Keep the dependencies and `uv.lock` current, keep the generated setup, UI, and product files, and list only variable names in `.env`. Set `status: ready` in `nanoagent.yaml`. Then, from your plugin folder, run:

```
uv sync --extra dev
uv run pytest
```

Fix all errors. Put your verifier tests in `tests/test_verifier.py`.

3.3 Publish and validate on RUSH Lab

Send your GitHub username to the course staff for an invitation, then sign in at `https://rush-lab.org` with the invited GitHub account.

The agent and your plugin run on your own Mac, in Mochi Lab, which runs in the background; you keep using RUSH Lab in your browser. To install Mochi Lab, open **Settings** → **AI**, copy the command shown there, and paste it in Terminal. It asks you to approve your computer in the browser once. Mochi Lab runs on macOS only (Apple silicon or Intel); if you do not have a Mac, contact the course staff. If Mochi Lab has no OpenRouter API key, RUSH Lab gives it one; you can instead save your own key from OpenRouter under the same pane. RUSH Lab sets the model.

Push the complete, tested plugin to a **public GitHub repository**. Open the Marketplace, select **Submit your artifact**, paste the repository URL, tick the statement, and select **Submit**. If the plugin is in a subfolder, use its `/tree/<branch>/<folder>` URL. Fix any reported errors and submit again.

Passing the publication checks confirms the package format only; it does not show that your task or verifier is correct.

Ask a workspace owner to add your plugin, then open it, ask its chat to run your task, and save the task verifier's verdict shown with the result. Have another student or a TA reproduce your setup and your correct- and incorrect-result checks from the README, and record any issues and fixes.

4 Questions

In your report, explain:

1. why your task is objectively verifiable;
2. how your environment is reproducible;
3. how the verifier handles incorrect outputs and boundary cases; and
4. what limitations remain.

5 What to Submit

Submit the GitHub URL and commit hash; your RUSH Lab plugin identifier; the complete source and lockfile; the tests; and a README with exact setup and execution instructions. Outside the plugin folder, include the inputs, validation logs, verdicts, and the independent reproduction record. Also submit a short report answering the questions above. TAs must be able to reproduce your results without manual fixes.